

Jupyter Notebook Execution Report

Name: Rodrigo Tapiador Cano

Date: April 04, 2026

Cell 1: ■ Markdown

Notebook de Repaso Final: Machine Learning Avanzado (Sesiones 14-19)

Bienvenido al "Infierno de los Datos". Hoy no te vamos a dar el dataset limpio ni te vamos a decir qué algoritmo usar.

Tienes dos casos de negocio reales y complejos. Tu misión es sobrevivir.

****Temas a evaluar:****

- * Limpieza de Datos (Nulos, Outliers).
- * Validación de Modelos y Detección de Data Leakage.
- * Interpretabilidad (SHAP) y Ética en IA.
- * Clustering (K-Means, MeanShift) y Reducción de Dimensionalidad (PCA/t-SNE).

Cell 2: ■ Markdown

Ejercicio 1: La Auditoría del Algoritmo de Contratación

Contexto

Una startup tecnológica ha estado usando una IA llamada "HireBot 3000" para filtrar currículums automáticamente.

* ****El Problema:**** Aunque el modelo tiene un 99% de acierto en predecir a quién contratan, RRHH ha notado que ****solo contratan hombres de ciertos códigos postales****.

* ****Tu Misión:**** Auditar el dataset, encontrar la "trampa" (Data Leakage) y descubrir los sesgos ocultos.

Tareas

1. Limpieza

- * Carga el dataset sucio.

- * Imputa los valores nulos en `Years\_Experience` y `Education\_Level` (cuidado con cómo lo haces).
- * Codifica las variables categóricas (`Gender`, `ZipCode`).

2. Detección de Data Leakage

- * Entrena un modelo rápido (Random Forest) para predecir `Hired` (Contratado: Sí/No).
- * Si tu Accuracy es > 95%, sospecha. Usa `feature\_importances\_` para ver qué variable está "haciendo trampa".

3. Clustering de Perfiles

- * Usa solo las variables demográficas y de experiencia.
- * Aplica **K-Means** para encontrar 4 tipos de candidatos.
- * Analiza los clusters: ¿Hay algún cluster que sea 100% hombres o 100% de un barrio rico?

4. Interpretabilidad

- * Usa **SHAP** sobre tu modelo corregido.
- * ¿Qué variable pesa más? ¿El `ZipCode`? ¿El `Gender`?
- * Redacta un informe ético: ¿Deberíamos seguir usando este modelo?

Cell 3: ■ Code

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.cluster import KMeans
import shap

# --- GENERACIÓN DE DATOS SUCIOS Y SESGADOS ---
def generate_hr_data(n=2000):
    np.random.seed(666) # El número de la bestia para datos malditos

    # Variables Base
    gender = np.random.choice(['M', 'F', 'NB', np.nan], n, p=[0.55, 0.40, 0.02, 0.03])
    experience = np.random.normal(5, 3, n)
    experience[experience < 0] = 0 # Corregir negativos
    # Introducir nulos en experiencia
```

```

experience[np.random.choice(n, 200, replace=False)] = np.nan

zip_code = np.random.choice([28001, 28002, 28050, 28999], n) # Barrio Rico vs
Barrio Pobre

interview_score = np.random.normal(7, 2, n)

score = (experience * 0.3) + \
(np.where(gender=='M', 10, 0)) + \
(np.where(zip_code==28001, 20, 0)) + \
np.random.normal(0, 2, n)

hired = np.where(score > 12, 1, 0)

interview_score = hired * 10 + np.random.normal(0, 1, n)

df = pd.DataFrame({
    'Gender': gender,
    'Years_Experience': experience,
    'ZipCode': zip_code,
    'Education': np.random.choice(['PhD', 'Master', 'Bachelor', np.nan], n),
    'Interview_Score': interview_score, # <--- DATA LEAKAGE!!!
    'Hired': hired
})
return df

df_hr = generate_hr_data()
print("Dataset de RRHH cargado con éxito. ¡Empieza la auditoría!")
df_hr.head()

```

Output:

```

Dataset de RRHH cargado con éxito. ¡Empieza la auditoría!

```

	Gender	Years_Experience	ZipCode	Education	Interview_Score	Hired
0	F	7.067659	28999	Master	0.210221	0
1	F	3.562925	28999	nan	-0.799074	0
2	F	8.201636	28002	PhD	-0.660457	0
3	F	8.358814	28001	nan	9.915297	1
4	NB	5.920068	28002	Master	0.692062	0

Ejercicio 2: El Cluster "Paciente Cero"

Contexto

Un hospital ha recopilado datos de análisis de sangre de 5.000 pacientes. No sabemos qué enfermedad tienen (no hay etiqueta `Diagnosis`), pero los médicos sospechan que hay **3 grupos claros** y un pequeño grupo de **anomalías peligrosas**.

Tareas

1. Preprocesamiento Robusto

- * Tienes muchas columnas numéricas (`Blood\_Pressure`, `Cholesterol`, `Glucose`...).
- * Escala los datos (StandardScaler es obligatorio en Clustering).

2. Reducción de Dimensionalidad (El Mapa)

- * Usa **PCA** para reducir a 2 dimensiones y graficar. ¿Ves grupos separados?
- * Prueba con **t-SNE**. ¿Mejora la visualización?
- * **Pregunta:** ¿Cuánta varianza explicada perdimos al bajar a 2D con PCA?

3. Clustering (El Diagnóstico)

- * Aplica **K-Means** con K=3 (lo que dicen los médicos).
- * Aplica **DBSCAN** (o MeanShift) para intentar detectar el "Ruido" o pacientes anómalos que K-Means ha forzado a entrar en un grupo.
- * Pinta los resultados sobre el gráfico de t-SNE.

4. Validación del Cluster

- * Calcula el **Silhouette Score** de tu K-Means. ¿Es un buen agrupamiento?
- * Analiza los centroides y ponles nombres médicos inventados (Ej: "Diabéticos", "Sanos", "Hipertensos").

Cell 5: ■ Code

```
from sklearn.datasets import make_blobs, make_moons
from sklearn.preprocessing import StandardScaler

# --- GENERACIÓN DE PACIENTES ---
def generate_medical_data():

X, y = make_blobs(n_samples=5000, centers=3, n_features=10, cluster_std=2.5,
random_state=42)
```

```

# Convertir a DataFrame con nombres médicos
cols = ['Blood_Pressure', 'Cholesterol', 'Glucose', 'Iron', 'Magnesium',
        'White_Cells', 'Red_Cells', 'Vitamin_D', 'Oxygen_Level', 'Heart_Rate']
df_med = pd.DataFrame(X, columns=cols)

anomalies = np.random.uniform(low=-15, high=15, size=(50, 10))
df_anomalies = pd.DataFrame(anomalies, columns=cols)

df_final = pd.concat([df_med, df_anomalies], ignore_index=True)

# Desordenar
df_final = df_final.sample(frac=1, random_state=42).reset_index(drop=True)

return df_final

df_patients = generate_medical_data()
print("Base de datos del Hospital cargada. (5050 Pacientes, 10 Métricas)")
print("No hay etiquetas. ¡Buena suerte encontrando los patrones!")

```

Output:

```

Base de datos del Hospital cargada. (5050 Pacientes, 10 Métricas)
No hay etiquetas. ¡Buena suerte encontrando los patrones!

```